

Week-10-11

DAY-67

1. Depth-First Search:

Recursive:-

```
def recursiveDFS(graph, node, visited = None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    if node in visited:
```

```
        return
```

```
    visited.add(node)
```

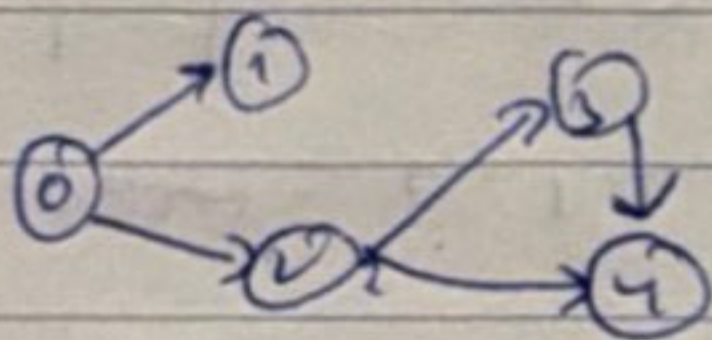
```
    print(node)
```

```
    for neighbor in graph[node]:
```

```
        recursiveDFS(graph, neighbor, visited)
```

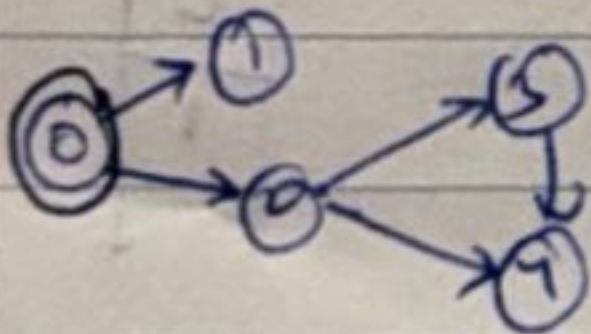
```
recursiveDFS(graph, 0, None)
```

graph = { 0: [1, 2], 1: [], 2: [3, 4], 3: [], 4: [] }



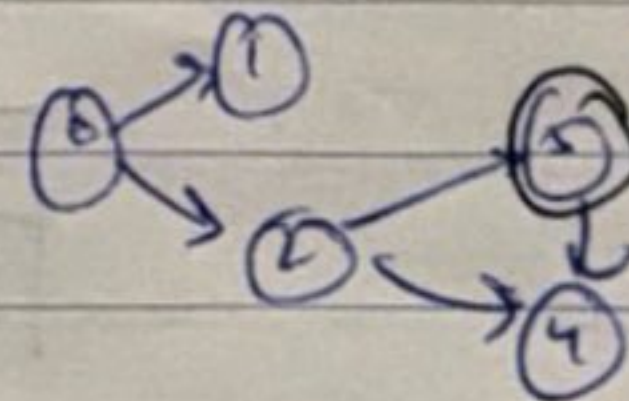
Visited = Set()

(i)



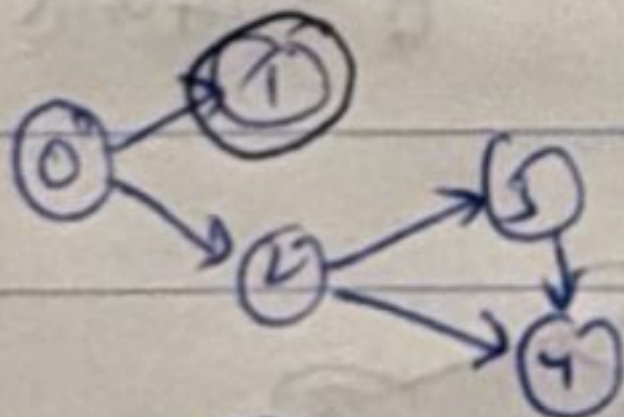
Visited = [0]

(iv)



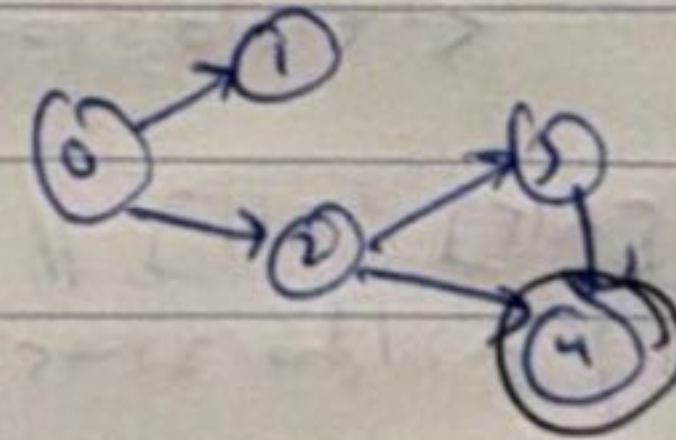
Visited = 0, 1, 2, 3

(ii)



Visited = [0, 1]

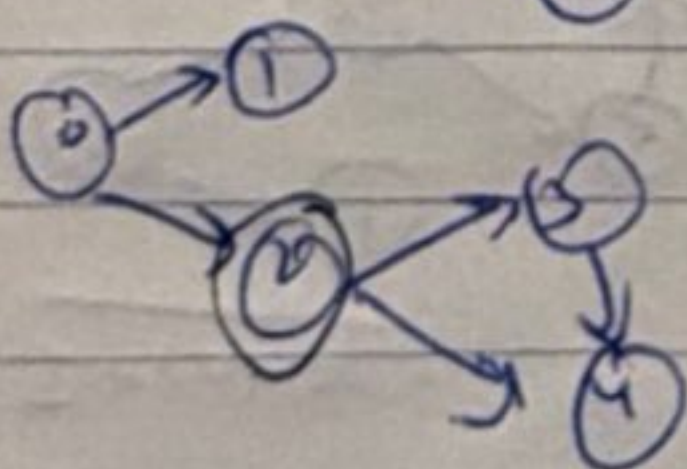
(v)



Visited = 0, 1, 2, 3, 4

Since 3 has no neighbors
2 neighbors 4

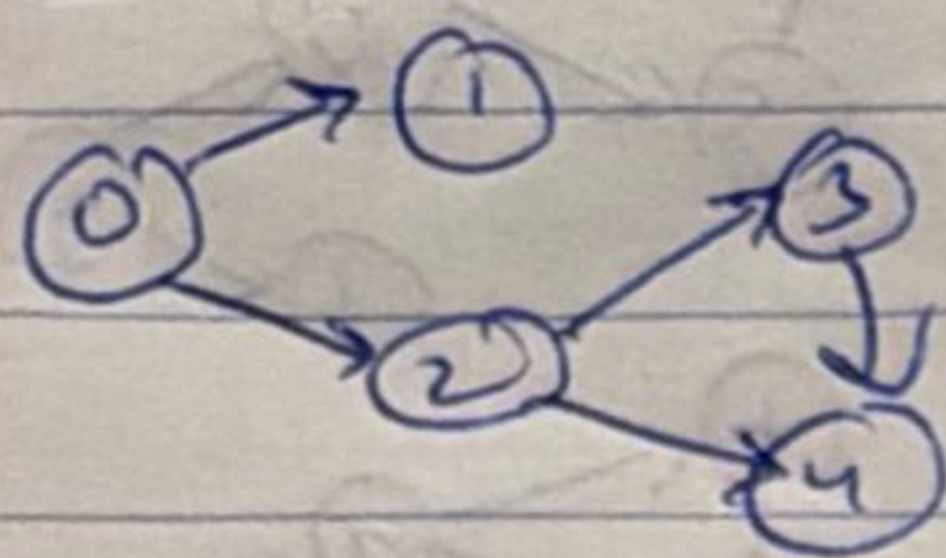
(iii)



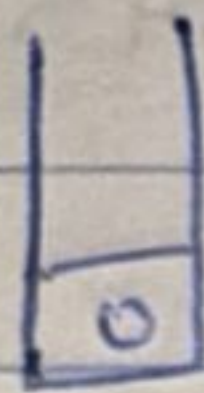
Visited = 0, 1, 2

Since 1 has no neighbors.

Iterative:



visited = set()



stack = [1, 2]

stack.extend([3, 4])

// [1, 2, 3, 4]

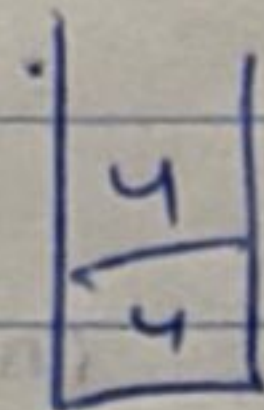
stack.append([3, 4])

// [1, 2, [3, 4]]

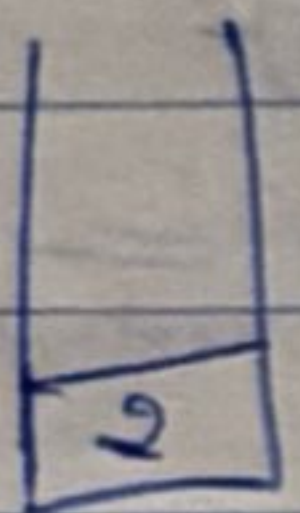
(i) visited = 0



(iv) visited = 0, 1, 2, 3

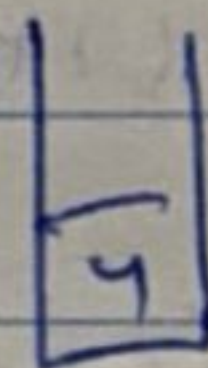


(ii) visited = 0, 1

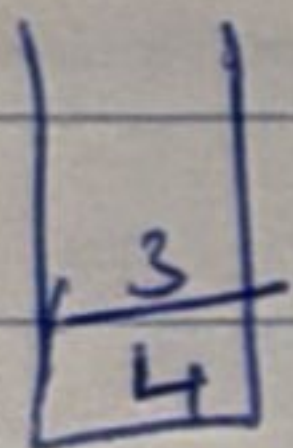


graph[1] = 1

(v) visited = 0, 1, 2, 3, 4



(iii) visited = 0, 1, 2



(vi) 4 in visited nothing to do

(v) stack is empty ← done.

BFS:-

```
def bfs (graph, start):
```

```
    visited = set()
```

```
    queue = deque([start])
```

```
    visited.add(start)
```

```
    while queue:
```

```
        node = queue.popleft()
```

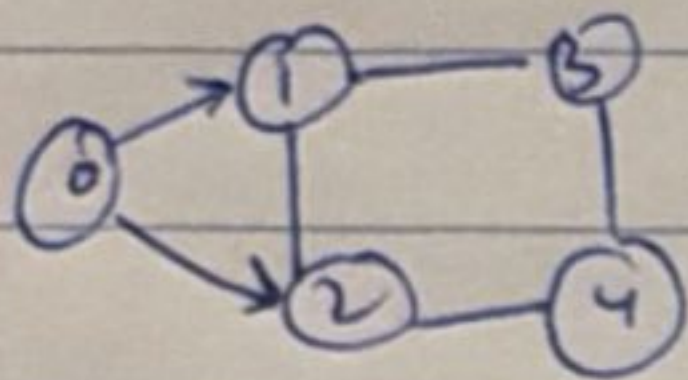
```
        print(node)
```

```
        for neighbour in graph[node]:
```

```
            if neighbour not in visited:
```

```
                visited.add(neighbour)
```

```
                queue.append(neighbour)
```



visited = {}

queue = [0]

node = 0; visited.add(0)

(1)

visited = 0

que = [1, 2]

(2)

visited = 0, 1, 2

queue = [3, 4]

(3)

visited = 0, 1, 2, 3, 4

que = []

(5) visited = 0, 1, 2, 3, 4

que = []

(4)

visited = 0, 1, 2, 3, 4

queue = []